

MASTERING AGENTIC AI · WEEK 2 PROJECT

FinRAG — Financial Document Intelligence

Retrieval-augmented question answering over SEC filings. Every answer carries a figure and a citation to the exact passage in the exact filing — or is refused.

Repository github.com/TatianaK-ai/FinRag

Corpus Latest 10-K and 10-Q for Apple, Microsoft and NVIDIA, from SEC EDGAR

Stack LangChain + LangGraph (Python), Pinecone, OpenAI, FastAPI

Contents Project documentation · Framework · Operating manual

FinRAG — Project Documentation

Week 2 · Project 2: Financial Document Intelligence Pipeline · Track 2 (code-heavy)

Built with LangChain + LangGraph (Python), Pinecone, OpenAI.

Companion documents: [FRAMEWORK.md](#) (the Part 1 framework) · [../out/REPORT.md](#) (generated evaluation) · [../README.md](#) (setup and architecture)

1. Project overview

What it does

An analyst asks a question about a company's SEC filing and gets the reported figure back **with the filing, section, and EDGAR link it came from** — or an explicit refusal when the corpus does not contain the answer.

```
$ python -m finrag.cli "What was NVIDIA's gross margin in fiscal 2026?"
```

```
NVIDIA's gross margin in fiscal 2026 was 71.1%, down 3.9 percentage points  
from 75.0% in fiscal 2025 [1].
```

```
[1] NVIDIA Corporation 10-K (filed 2026-02-25)  
    Item 7. Management's Discussion and Analysis  
    https://www.sec.gov/Archives/edgar/data/1045810/...
```

Why the refusal path is the centre of the design

The brief says the "I don't know" path matters more than the happy path. In financial Q&A that is not a stylistic preference — a confidently wrong figure is worse than no figure, because an analyst acts on it. So the graph has **four independent gates**, and any one of them can decline:

```

START └─> scope ──ambiguous──> clarify ─> END
      └─> retrieve          (runs concurrently with scope)
          |
          gate ──guidance request──> refuse ─> END
          |
          rerank ─> grade ──insufficient──> refuse ─> END
              |
              generate ──false premise──> refuse ─> END
                  |
                  ─not answerable──> refuse ─> END
                  |
                  verify ─ungrounded──> refuse ─> END
                      |
                      ─wrong entity──> refuse ─> END
                      |
                      ─answers a
                      |
                      different fact──> refuse ─> END
                      v
                      END

```

Measured across three repetitions: **100% correct refusal, in every run.**

Headline results

metric	result
Retrieval hit@6 (with reranking)	1.000
Wrong figures delivered	0 / 51 judged answers
Faithfulness	1.000 (range 1.000–1.000 across 3 runs)
Correct refusal (out-of-corpus questions)	100% (100–100%)
Clarification (underspecified questions)	100%
Answer rate	100% (100–100%)
False refusal	0.0%
Latency p50	5,445 ms (target < 6,000 ms)

The most useful single comparison in the whole project: the same retrieval **without** reranking produced **4 wrong figures in 42 answers**; **with** reranking, **0 in 51**.

Every figure above is from `out/REPORT.md`, three repetitions on the shipped arm. Numbers quoted inside §3 and §4 below are the measurements taken *at the time each change was made*, so they trace the development sequence rather than the current build.

2. Datasets used

Six filings pulled live from the **SEC EDGAR** API — public regulatory disclosures, fetched under SEC's fair-access policy (identifying `User-Agent`, request spacing well under the 10 req/s limit, results cached to disk so re-runs do not re-fetch).

Document	Form	Filed	Period
AAPL_10K_2025-10-31	10-K	2025-10-31	FY2025 (ended Sep 2025)
AAPL_10Q_2026-07-31	10-Q	2026-07-31	Q3 FY2026
MSFT_10K_2026-07-29	10-K	2026-07-29	FY2026 (ended Jun 2026)
MSFT_10Q_2026-04-29	10-Q	2026-04-29	Q3 FY2026
NVDA_10K_2026-02-25	10-K	2026-02-25	FY2026 (ended Jan 2026)
NVDA_10Q_2026-08-26	10-Q	2026-08-26	Q2 FY2027

After cleaning: 1,382 structural blocks · **348 tables** · ~1.33 M characters · **847 kB of inline XBRL removed.**

Three issuers with deliberately misaligned fiscal calendars (Apple ends September, Microsoft June, NVIDIA January), which makes cross-company comparison questions genuinely hard rather than cosmetic.

Chunking

strategy	chunks	mean chars	stdev
fixed	1,619	871	393
semantic	1,253	1,060	725

Both handle tables **identically** (atomic, split by row group with the header repeated when oversized). That is deliberate: it holds table parsing constant so the comparison measures prose segmentation, not table handling.

3. Prompts and agent instructions

Five model calls, each with a narrow job. Full text lives in `finrag/graph/rag.py` and `finrag/retrieval/rerank.py`.

Scope gate (`gpt-4.1-mini`)

Decides whether the question says *which company* it means — judged on the **question text alone**. `scope` and `retrieve` run concurrently from `START` and join at `gate`, so scope never sees the passages and cannot be biased by whichever issuer retrieval happened to surface.

"isAmbiguous=true ONLY when the question refers to a company merely as 'they', 'the company', or not at all... A company named but ABSENT from the corpus (e.g. Tesla) is NOT ambiguous — the subject is perfectly clear."

Reranker (`gpt-4.1-mini` , LLM cross-encoder)

Scores each passage 0–10. Returns **scores only, no rationale** — see §5, that decision alone cut 6.4 s of latency.

"Be strict about entity (which company) and period (which fiscal year/quarter). A passage about the right metric for the WRONG company or year scores at most 4."

Generator (`gpt-4.1-mini`)

Answers from numbered passages with inline `[n]` citations.

"Use ONLY the numbered passages. Never use outside knowledge, never estimate, never interpolate a figure that is not printed... Reproduce figures exactly as printed, including units and scale (the tables are usually 'in millions'). Do not re-scale or round."

"PREMISE. Set premiseHolds=false when the question assumes something these filings do not support... A figure is NOT an answer merely because it shares a year with the question."

It must also name `requestedFact` — the exact metric, company and period — **before** answering. Forcing that statement first is what stops it settling for an adjacent figure.

Verifier (`gpt-4.1-mini`)

Reads the answer back against its passages, checking two things: every claim supported, **and** that the answer addresses the metric, company and period actually asked about. Grounding alone is not sufficient.

Judge (`gpt-4.1` — deliberately stronger than the generator)

Grades faithfulness, correctness, relevance and citation validity, so the evaluation is not the generator marking its own homework.

Deterministic gates (no model involved)

Three checks run in code, because the models demonstrably failed them:

- **Guidance guard** — a 10-K reports what happened; it does not publish forward guidance. Requires a projection word *and* a period beyond the corpus horizon, while exempting real accounting terms ("projected future minimum lease payments").

- **Entity grounding** — any named entity in the question that appears in **no cited passage** blocks the answer. This is \$5's most important fix.
- **Index freshness** — a content fingerprint binds vectors to the chunks they were built from; a mismatch throws rather than silently serving wrong text.

4. Iterations tried

Ordered roughly as they happened. Each was driven by a measurement, not a hunch.

Ingestion

1. **Naive .text() extraction** → produced runs like `0000320193us-gaap:CommonStockIncludingAdditionalPaidInCapitalMember2022-09-24`. Filings are inline-XBRL; a large share of the "text" is machine tagging. Stripping `<ix:header>` removed **847 kB** and took leakage to zero.
2. **Column-wise table cleanup** → failed. SEC tables are ragged *per row*: the `$` sits in its own cell but is only emitted on the first and last row of a block. Switched to **row-wise compaction**, which collapses every data row to the same arity.
3. **Em-dash bug** — a lone `–` (the filing's "nil" marker) was glued onto the next figure, turning `| $209,586 | 4% | $201,183 | – | $200,583 |` into `| $209,586 | 4% | $201,183% | –$200,583 |`. **Two figures corrupted at once, silently.** Now treated as a value, not punctuation.
4. **Headerless tables** — Apple's debt-maturity schedule starts straight at `| 2026 | $12,393 |`. Treating row 0 as headings lost a row *and* made the caption claim the table "covers fiscal year 2026" when it runs to Thereafter.

Retrieval

5. **Dense only** → nDCG 0.279. Embeddings cannot tell `$416,161` from `$391,035`.
6. **BM25 only** → nDCG 0.318 at **~1 ms**. Cheaper *and* better than dense on this corpus, because in filings the figure *is* the answer.
7. **Hybrid (RRF)** → 0.385. Chosen over score normalisation because cosine and BM25 scores are not comparably scaled and shift per query; RRF needs only ranks.
8. **Dense and sparse were indexing different text** — embeddings got a provenance prefix, BM25 got raw chunk text. Since a financial table names no company, BM25 was structurally blind to which issuer a table belonged to and dragged fused ranking *below* dense-only.
9. **Table captions** — the biggest single retrieval win. Tables lose to prose in *both* retrievers: a grid of numbers embeds nowhere near a natural-language question, and BM25 rewards prose that repeats the query's words. The gap was vocabulary — the table says `2026`, the question says "fiscal year 2026". Each table now carries a caption built deterministically from its own row labels and column headers. **BM25 hit@6 0.412 → 0.765**; the Microsoft revenue figure moved from **hybrid rank 30** → **rank 3**.

10. **Per-entity retrieval** — for "Microsoft **or** NVIDIA", a single pass returned **27 NVDA / 2 MSFT / 1 AAPL** and the Microsoft figure was absent entirely. Reranking cannot recover what retrieval never surfaced. Comparison questions now get one pass per issuer with an equal share of the window.
11. **Per-issuer floor** — with three issuers dividing a flat 30-candidate window, each got 10, and the table holding Microsoft's total growth fused to **rank 11** — outside by one. Added a floor of 14 per issuer.

The graph

12. **Refusal designed first**, then extended twice *because evaluation caught it failing*:
 - **premise** — asked for FY2027 revenue guidance, the system answered **\$7,652 million** from a 2027 lease-maturity column. Grounded, cited, and wrong.
 - **verify.answersTheQuestion** — grounding alone is not enough; an answer can quote a passage perfectly and be about a different fact.
13. **Clarification in the generator** → **didn't work**. Shown six NVIDIA passages, it reasonably concluded the question was about NVIDIA. Ambiguity is a property of the *question*, so **scope** became its own node, judged on the question text alone and never shown the retrieved passages.
14. **Scope on gpt-4.1-nano** → **reverted**. Saved 260 ms and broke behaviour classification: correct refusal fell **100% → 25%**. Documented in **config.py** so it is not re-attempted.
15. **Entity grounding gate** — see §5.
16. **Schema field descriptions are part of the prompt**. Structured-output schemas are serialised into the request, so the one-line description on each field steers the model as much as the system prompt does. Shipping **ScopeOut** with bare **bool** fields made "*which of the three companies grew revenue fastest*" read as ambiguous in **3 runs out of 3** — nothing told the model that deliberately spanning all issuers still identifies the subject. Restoring the system prompt alone did **not** fix it; restoring the field descriptions did. **Answer rate 94.1% → 100%**.

Latency

17. **Rerank rationales removed** — asking for a one-line reason per passage meant 20 rationales generated token by token: **8.9 s** median. Scores only → **2.5 s**.
18. **Concurrent rerank batches** — 5.3 s → 2.1 s. (A *smaller model* was tried and measured **slower**; output length was the bottleneck, not model size.)
19. **scope parallel with retrieve** — two independent calls were running in series for no reason. **7,377 ms → 5,723 ms p50**.

5. Learnings and observations

Grounding is not sufficient — the single most important finding

Asked "What price did NVIDIA pay to acquire Intel's foundry business?" — an acquisition that never happened — the system answered **\$17.0 billion**.

It was not hallucinating. Retrieval surfaced a **real NVIDIA acquisition disclosure** (\$13.0 bn at closing + \$4 bn payable) that mentions neither Intel nor foundry. Every gate passed it correctly *on its own terms*:
`grounded: true` (the figures really are in the passage), `answersTheQuestion: true` (the passage really is about an acquisition price). Nothing asked "*is it about Intel?*"

Across three runs it slipped through twice.

The fix could not be another prompt. A named entity the question asks about that appears in **no cited passage** cannot be what the answer is about — that is decidable without a model, so it is now decided without one. Validated against all 24 questions with real retrieval: **0 false positives**, catches the failure. Then **five consecutive live runs: refuse, refuse, refuse, refuse, refuse**.

The general lesson: **when a model gate fails a check that code can perform, move the check into code**. The same reasoning produced the guidance guard and the index-freshness assertion.

Single-run evaluation is not evaluation

The gates are model calls and do not settle on one answer. Three runs on **identical code**, measured while the refusal gates were being built:

	run 1	run 2	run 3
correct refusal	75%	100%	75%
failures	q17, q24	none	q17, q24

Run 2 was a perfect score. Reporting it alone would have been true and misleading.

Worse, I reasoned badly from that middle run: after two runs I concluded q24 "did not reproduce" and was non-determinism — then run 3 showed it failing again. **I made an n=2 mistake immediately after explaining why n=1 conclusions are unsafe**. The eval now supports `EVAL_REPEATS=3` and the report prints ranges plus an "unstable questions" column, so a single lucky run cannot be quoted as the result.

Metrics can flatter the system that is least useful

Faithfulness is measured only on answers the pipeline chose to emit — `verify` refuses anything it judges ungrounded, and only non-refused answers reach the judge. So **it can be raised by refusing more**, and the weakest arm posted the best score while failing 41% of answerable questions.

The report now prints **answer rate first**, faithfulness **with its denominator**, and a **counted wrong-figure rate** rather than an averaged score that buries single failures.

Two related traps found in our own reporting:

- **"p95" was literally the maximum**. At n=17, `floor(0.95 × 17) = 16` — the last index. It was reported against a 6,000 ms target as though it were a percentile.

- **The failure-analysis filter never checked correctness**, so an answer that was faithful to its passages while reporting the wrong figure could not appear in it — precisely the failure the project exists to catch.

Small differences on a small set are not results

With 17 answerable questions, one question is worth **5.9 points**. An early draft declared "*Winner on raw dense retrieval: fixed (+0.5 pts)*" — the two arms differed on **exactly one question**, and under reranking they tie **0 wins / 0 losses / 17 ties**.

The report now computes paired win/loss/tie counts and refuses to name a winner unless the record is decisive. On this data the honest statement is: **the two chunking strategies are indistinguishable at this sample size**.

Tests can be green and worthless

Three separate times a test asserted something other than what its name claimed:

- A test for "the guidance guard stays narrow" called `retrieve()` **directly** — a layer the guard does not run in. Green, zero coverage, and cited as evidence the guard was safe.
- A test for "reranking survives a partial failure" used 3 passages (one batch), failed nothing, and asserted only `Array.isArray(hits)` — true for every possible return value including the bug it was named for.
- A test for "per-entity retrieval reports timings" passed on the zero-initialised object.

All three were rewritten to fail when the bug is present, verified by reproducing the old behaviour first. The suite has since caught **four** intentional behaviour changes, which is the useful kind of failure.

Derived data needs to be bound to its source

Running the mock-backed offline build after a real build replaced `chunks.semantic.json` with the mock's chunking. The vector store still held vectors keyed to the *real* chunk ids, so lookups returned ids that no longer existed on disk and were **silently dropped**. It read as a catastrophic retrieval regression (`semantic/dense` hit@6 0.529 → 0.118) rather than as corruption. Measured afterwards: **466 fixed and 953 semantic ids whose stored text differed from what the chunk store served**.

Now: tests write to an isolated `DATA_DIR`, a content fingerprint binds each index to its chunks, and a mismatch throws on first retrieval instead of degrading quietly.

Reranking is where the quality is

Same retrieval, same corpus, three repetitions:

	without rerank	with rerank
hit@6	0.882	1.000
nDCG@6	0.385	0.796
MRR	0.597	0.971
wrong figures	4 / 42	0 / 51
answer rate	82.4%	100%
p50 latency	5,019 ms	5,445 ms

A bi-encoder scores query and passage independently, so it can only measure topical similarity. A cross-encoder reads the pair together and can act on "fiscal 2026" as a constraint — exactly the distinction that matters where every issuer files a structurally identical document each year.

6. Honest limitations

These are in the generated report too, not only here.

- **The golden set is small and was revised after seeing results.** 24 questions (17 answerable), one author. Evidence patterns were broadened and one metric redefined *after* observing failures — each change defensible, but **all moved scores upward and none moved one down**. The refusal gates were also tuned against these same questions. **Treat the absolute numbers as training-set numbers.**
- **Coverage is narrow.** 16 of 17 answerable questions target a 10-K, and all are income-statement figures. Nothing tests balance sheet, cash flow, footnotes, or the 10-Qs. Real users will ask those immediately.
- **Faithfulness is conditional on answering** — read it beside answer rate and wrong-figure count, never alone.
- **Latency p50 meets the 6 s target; the tail does not.** p95 is omitted because at this sample size it is an extreme order statistic contaminated by API retries.
- **Entity substitution is closed; other substitutions are not.** A wrong-*period* answer with the right entity would still pass every gate.
- **The rate limiter is in-memory and per-process.** Behind a load balancer, each instance gets its own budget.
- **Cold start is ~14 s** on the first request after boot.

7. How to reproduce

```
python -m venv .venv && .venv/Scripts/activate      # Windows; `source .venv/bin/activate` on Linux/Mac
pip install -r requirements.txt
cp .env.example .env                                # add OPENAI_API_KEY, PINECONE_API_KEY, SEC_USER_AGENT

python -m finrag.build                               # fetch → clean → chunk (both strategies)
EVAL_REPEATS=3 python -m finrag.evals.run            # full experiment matrix with run-to-run r
python -m finrag.evals.report                        # renders out/REPORT.md
python -m finrag.server                             # chat UI on :3000
```

Everything also runs **offline with no credentials** against a bundled mock — `python tests/offline.py` builds the corpus and runs 32 behavioural tests in about 40 seconds. Offline numbers prove the wiring, never the quality.

Set `APP_API_KEY` **before exposing the server.** Without it the process binds to `127.0.0.1` only and refuses off-box connections, because every `/api/ask` call spends real API credit.

Part 1 — The RAG Framework

The one-liner

My RAG app helps **equity and credit analysts** answer **specific quantitative and disclosure questions** from **the latest 10-K and 10-Q filings of AAPL, MSFT and NVDA (6 documents, ~1.3 M characters, 348 tables, sourced live from SEC EDGAR)** in a **web chat UI and a CLI** with **≥95% faithfulness** and **100% correct refusal on out-of-corpus questions**, at **p95 latency under 6 seconds**.

Against the three rules in the brief:

- **The corpus is named specifically.** Not "financial documents" — six identified filings, fetched by CIK from EDGAR, re-fetchable with one command, with their accession numbers recorded in the manifest. Every chunking and retrieval decision below follows from what those documents actually are: long, structurally repetitive, table-dense, and near-identical across issuers and years.
 - **Faithfulness is the headline metric, not relevance.** The target is 95% of claims traceable to a retrieved passage, graded by a model stronger than the generator. Relevance is tracked too, but an answer that sounds right and cites a figure the filing never printed is the specific failure this system exists to prevent.
 - **Latency has a ceiling.** p95 under 6 s end-to-end, including reranking and the grounding check. That budget is what stops retrieval from growing a third and fourth stage just because each one nudges nDCG.
-

The framework

Field	Decision
Use case	An analyst asks for a specific reported figure or disclosure ("What was NVIDIA's FY2026 gross margin?", "How did Apple's Greater China revenue move?") and needs the number with the filing and section it came from. Surfaces: a browser chat UI, a CLI, and a JSON HTTP API for embedding elsewhere.
Corpus	6 filings — latest 10-K and 10-Q each for Apple, Microsoft, NVIDIA — pulled from the SEC EDGAR submissions API by CIK. English, inline-XBRL HTML, ~1.3 M cleaned characters, 348 tables, 1,382 structural blocks. The SEC is the source of truth; nothing is hand-edited after download.
Ingestion + cleaning	<code>data.sec.gov/submissions/CIK*.json</code> → primary document HTML → strip <code><ix:header></code> / <code><ix:hidden></code> (847 kB of XBRL tagging that would otherwise appear as text like <code>0000320193us-gaap:CommonStock...</code>) → drop <code><script></code> / <code><style></code> → convert every <code><table></code> to markdown with per-row compaction so <code>\$ / %</code> /parens re-attach to their figures → drop page-footer boilerplate (467 lines) → detect <code>Item N.</code> headings to label each block with its section, remembering each item's title so a bare running header (<code>Item 7</code> on its own) cannot strip it. Entities are decoded and NBSP/smart quotes normalised. Sections become chunk metadata and drive citations.

Every table chunk also carries a **synthesized caption** built deterministically from its own row labels and column headers — *"Financial table from Microsoft Corporation (MSFT) 10-K. Covers fiscal year 2026, fiscal year 2025. Reported line items: Revenue, Gross margin, ..."*. Tables otherwise lose to prose in both retrievers: a grid of numbers embeds nowhere near a natural-language question, and BM25 rewards prose that repeats the query's words while a table repeats nothing. The concrete gap was vocabulary — the table says `2026`, the question says "fiscal year 2026". Captioning roughly doubled BM25 hit@6 (0.412 → 0.765). |

| **Ingestion + freshness** | `python -m finrag.ingest.run` re-fetches and re-cleans; raw HTML is cached on disk so re-runs are free. Cadence matches the filings themselves: quarterly for 10-Qs, annually for 10-Ks, so a weekly cron is more than sufficient. Freshness SLA: indexed within 24 h of a filing appearing on EDGAR. Every chunk carries `filingDate` and `periodOfReport`, so staleness is visible in the answer rather than hidden. |

| **Chunking + embedding** | Two strategies, built and measured against each other. **Fixed**: recursive character splitting at 1,200 chars / 200 overlap (~300 tokens) → 1,620 chunks, stdev 393. **Semantic**: sentence embeddings with a 1-sentence context buffer, cut where consecutive cosine distance exceeds the document's 90th percentile, bounded to 350–2,000 chars → 1,252 chunks, stdev 724. Tables are atomic in **both** (split by row group with the header repeated when oversized) so the comparison isolates prose segmentation. Embeddings: `text-embedding-3-small` @1536d — 8 k context comfortably covers a chunk, and the whole two-strategy corpus embeds for well under a cent. Chunk size and model were chosen together: 1,200 chars is small enough that a chunk carries one claim, large enough that a financial sentence keeps its qualifying clause. |

| **Retrieve** | Pinecone serverless (cosine, one namespace per chunking strategy) for dense, plus a from-scratch BM25 index for sparse. Both index the **same** provenance-prefixed text — indexing different representations silently crippled sparse retrieval on tables. Fused with Reciprocal Rank Fusion (k=60, equal weights) — chosen over score normalisation because cosine and BM25 scores are not comparably scaled and shift per query, whereas RRF needs only the ranks. Retrieve top-40 from each, fuse, rerank the top 30 in concurrent batches, keep 6 for the generator. A local brute-force vector backend (`VECTOR_BACKEND=local`) mirrors the same interface so the project runs and the evals reproduce without a Pinecone account. |

Why hybrid, concretely

Dense retrieval and BM25 fail in opposite directions on this corpus, which is the entire argument for fusing them:

- **Dense misses exact tokens.** Embeddings place "revenue grew" and "revenue declined" close together, and cannot distinguish `$416,161` from `$391,035` — they are the same shape of thing. In filings the figure *is* the answer.
- **BM25 misses intent.** "How profitable is Apple's services business?" shares no useful term with the table headed `Gross margin` that answers it.
- **Both miss period.** Every issuer files a structurally identical document each year. The passage for FY2025 and the one for FY2024 differ by a handful of digits. Neither retriever alone reliably picks the right one — which is what the cross-encoder reranker is for, since it reads query and passage *together* and can act on "fiscal 2026" as a constraint rather than as a bag of tokens.

The refusal path, designed first

The brief's point — that the "I don't know" path matters more than the happy path — is taken literally here: the graph was built refusal-first, and there are **four independent gates** before any answer is returned. Two of them (`premise` and the question-match half of `verify`) were added *because* evaluation caught them missing.



1. **scope** — does the question even say which issuer it means? Judged on the question text alone: **scope** and **retrieve** fan out from **START** in parallel and join at **gate**, so **scope** never sees the passages and its verdict cannot be biased by whichever issuer retrieval happened to surface. Underspecified → **clarify**, which asks back. (Folding this into the generator did *not* work: shown six NVIDIA passages, it reasonably concluded the question was about NVIDIA.)
2. **grade** — is the retrieved evidence good enough to attempt an answer? When reranking has run, its calibrated 0–10 relevance is reused (no second round-trip); otherwise an explicit LLM grader decides. Best passage below 4/10 → refuse.
3. **premise** — does the question assume something the filings do not report? The generator must name the fact requested *before* answering, which is what stops it accepting a figure that merely shares a fiscal year. Asked for FY2027 revenue guidance, it previously answered from a 2027 lease-maturity figure — grounded, cited, and wrong.
4. **verify** — reads the answer back against its passages, checking two things: every claim supported, **and** that the answer addresses the metric, company and period actually asked about. Grounding alone is not sufficient.

A refusal is never a bare "I don't know": it states *why*, lists the closest material it did find, and names the corpus boundary. Seven of the 24 golden questions exercise this path (four that must be refused, three that must be clarified) — including one deliberately nasty case (Microsoft CEO compensation) where the section heading retrieves beautifully but contains no figures, because Item 11 is incorporated by reference to the proxy statement.

Success criteria

Metric	Target	Measured	Where it is measured
Faithfulness	≥ 0.95	1.000 (n=51)	LLM judge, <code>out/REPORT.md</code> §5. Conditional on answering - measured only on answers the pipeline emitted, so it must be read beside answer rate and wrong-figure count
Correct refusal on out-of-corpus questions	100%	100%	4 golden questions, §5 - one question moves this 25 points
False refusal on answerable questions	≤ 10%	0.0%	17 golden questions, §5
Retrieval hit@6	≥ 0.90	1.000	Golden-set evidence matching, §2–4
p95 end-to-end latency	< 6,000 ms	not demonstrated (p50 5,445 ms)	§5

Measured over three repetitions of the shipped arm (`fixed/hybrid+rerank`), plus **0 wrong figures in 51 judged answers** - the outcome the four gates exist to prevent. Every target except the latency tail is met, and the caveat under

§"Honest limitations" in `PROJECT.md` applies: these are training-set numbers on a 24-question set the gates were tuned against.

On the latency ceiling. The shipping pipeline makes four sequential model calls (scope → rerank → generate → verify). Two rounds of tuning brought the median under budget — dropping per-passage rationales from the reranker (8.9 s → 2.5 s), splitting the rerank window into concurrent batches (5.3 s → 2.1 s), A third attempt - running the scope gate on a cheaper model - was reverted after it broke behaviour classification. p95 is not demonstrated to be met, and at this sample size it is an extreme order statistic rather than a percentile, so §5 reports p50 and says so.

FinRAG — Financial Document Intelligence

A retrieval-augmented question-answering system over public SEC filings (Apple, Microsoft, NVIDIA — 10-K and 10-Q). It answers questions with a figure and a citation back to the exact passage in the exact filing, and **refuses** when the filings do not support an answer.

Built with LangChain + LangGraph (Python), Pinecone, OpenAI.

Full write-up: [docs/PROJECT.md](#) · framework:

[docs/Framework.md](#) · latest evaluation: [out/REPORT.md](#) .

```
$ python -m finrag.cli "What was NVIDIA's gross margin in fiscal 2026?"
```

```
NVIDIA's gross margin in fiscal 2026 was 71.1% [1].
```

```
[1] NVDA 10-K filed 2026-02-25 — Item 7. MD&A  
https://www.sec.gov/Archives/edgar/data/1045810/...
```

Setup

```
python -m venv .venv  
.venv/Scripts/activate          # Windows; source .venv/bin/activate elsewhere  
pip install -r requirements.txt  
cp .env.example .env            # then fill in the keys
```

`.env` needs `OPENAI_API_KEY`, and `PINECONE_API_KEY` unless you set `VECTOR_BACKEND=local`. `SEC_USER_AGENT` must be a real contact string — EDGAR rejects anonymous automated traffic.

Build the index

```
python -m finrag.build          # ingest -> chunk (both strategies) -> embed -> index
```

Raw filings are cached under `data/raw/`, so a rebuild does not re-hit EDGAR.

Ask

```
python -m finrag.cli "What were Apple's total net sales in fiscal 2025?"
python -m finrag.server # chat UI on http://127.0.0.1:3000
```

Set `APP_API_KEY` **before exposing the server.** Without it the process binds to `127.0.0.1` only and refuses off-box connections — every `/api/ask` call spends real API credit, so an open endpoint is a billing liability before it is a data one.

Evaluate

```
EVAL_REPEATS=3 python -m finrag.evals.run # experiment matrix, 3 repetitions
python -m finrag.evals.report # renders out/REPORT.md
```

Stage 1 sweeps retrieval (2 chunking strategies × 3 modes, plus reranking); stage 2 runs the full graph end to end and judges the answers with a *stronger* model than the one that generated them. `EVAL_REPEATS` matters: the gates are model calls and do not settle on one answer, and single-run figures have been wide enough to change conclusions.

Test

```
python tests/offline.py # builds the corpus against a mock API, then runs pytest
pytest # tests only, assuming an index already exists
```

The offline runner needs **no credentials and spends nothing**: a bundled mock OpenAI server (`tests/mock_openai.py`) serves hashed bag-of-words embeddings and schema-shaped completions, so the whole pipeline runs end to end. It builds into `DATA_DIR=data-test` so it can never overwrite a real index. Offline numbers prove the wiring, never the quality.

The `tests` badge above runs `pytest` on GitHub Actions (Python 3.10 and 3.12). CI covers the deterministic layers only — cleaning, tokenising, fusion, the scope and guidance guards, auth and rate limiting. The graph and retrieval tests skip there, because they need an index built from `data/raw/`, which is not in the repository. `python tests/offline.py` is the full 32-test gate.

Layout

finrag/	
config.py	models, retrieval constants, corpus, paths
build.py	ingest -> chunk -> index in one command
ingest/	EDGAR fetch, XBRL stripping, table-aware cleaning
chunking/	fixed-size and semantic strategies (shared table handling)
store/	BM25, local vectors, Pinecone, index freshness
retrieval/	hybrid dense+sparse with RRF, LLM cross-encoder rerank
graph/rag.py	the LangGraph state machine and its four refusal gates
evals/	two-stage harness, LLM judge, report renderer
middleware/guard.py	auth and rate limiting
server.py, cli.py	FastAPI surface and command line
eval/questions.json	the golden set
tests/	pytest suite + mock OpenAI server
docs/	project write-up and framework